# Especificação – Linguagem de Desenho (Fase 1)

Laboratório de Programação Avançada II – Compiladores

## 1. Descrição da linguagem

Linguagem de comandos para descrever desenhos 2D. Um programa começa com a declaração da tela (`canvas`), seguida de zero ou mais formas e grupos. Formas aceitam modificadores opcionais (cor do traço, cor do preenchimento, espessura). Grupos aceitam modificadores opcionais (rotação, deslocamento), delimitam seu conteúdo com `{ }` e podem ser aninhados em qualquer profundidade. Comentários começam com `//` e vão até o fim da linha.

| Comando | Forma | Argumentos |
|---|---|---|
| Tela | `canvas L A` | largura, altura |
| Reta | `segmento x1 y1 x2 y2` | dois pontos |
| Círculo | `circulo cx cy r` | centro e raio |
| Retângulo | `retangulo x y L A` | canto superior esquerdo, largura, altura |
| Grupo | `grupo [modificadores] { ... }` | formas e outros grupos |

Modificadores de forma (após os argumentos, em qualquer ordem): `cor_traco COR`, `cor_fundo COR`, `espessura N`.
Modificadores de grupo (entre `grupo` e `{`, em qualquer ordem): `rotacao N`, `deslocamento N N`.

Cores: `vermelho`, `azul`, `verde`, `preto`, `branco`, `amarelo`, `laranja` ou hexadecimal `#RRGGBB`.

### Exemplo 1

```
canvas 300 200
// casa simples
retangulo 100 100 100 80 cor_fundo amarelo cor_traco preto espessura 2
segmento 100 100 150 60 cor_traco vermelho espessura 3
segmento 150 60 200 100 cor_traco vermelho espessura 3
grupo deslocamento 20 20 {
  circulo 0 0 15 cor_fundo laranja
  grupo rotacao 45 {
    segmento 0 0 25 0 cor_traco #FF8800
  }
}
```

### Exemplo 2

```
canvas 100 100
grupo { }
grupo rotacao 30 rotacao 60 deslocamento 10 10 {
  circulo 50 50 20 cor_traco azul cor_traco verde
}
```

(usa grupo vazio, modificadores repetidos e modificadores de grupo em sequência)

## 2. Definição léxica

Os tokens são testados na ordem da tabela; vence o primeiro que casar na posição atual.

| Token | Expressão regular | Descrição |
|---|---|---|
| ESPACO *(ignorado)* | `[ \t\r]+` | espaços e tabulações |
| NOVA_LINHA *(ignorado)* | `\n` | apenas incrementa o contador de linha |
| COMENTARIO *(ignorado)* | `\/\/[^\n]*` | de `//` até o fim da linha |
| CANVAS | `canvas\b` | palavra-chave |
| SEGMENTO | `segmento\b` | palavra-chave |
| CIRCULO | `circulo\b` | palavra-chave |
| RETANGULO | `retangulo\b` | palavra-chave |
| GRUPO | `grupo\b` | palavra-chave |
| COR_TRACO | `cor_traco\b` | palavra-chave |
| COR_FUNDO | `cor_fundo\b` | palavra-chave |
| ESPESSURA | `espessura\b` | palavra-chave |
| ROTACAO | `rotacao\b` | palavra-chave |
| DESLOCAMENTO | `deslocamento\b` | palavra-chave |
| COR | `(vermelho\|azul\|verde\|preto\|branco\|amarelo\|laranja)\b` ou `#[0-9a-fA-F]{6}\b` | cor nomeada ou hexadecimal |
| NUM | `(0\|[1-9][0-9]*)\b` | inteiro sem sinal |
| ABRE | `\{` | início de grupo |
| FECHA | `\}` | fim de grupo |

O `\b` ao final das palavras-chave impede que uma palavra maior (ex.: `circulos`) seja quebrada em `circulo` + `s`.

**Erro léxico:** se nenhuma regra casar, o lexer registra erro com a linha e o trecho inválido (do ponto atual até o próximo espaço em branco) e continua depois dele. Se houver qualquer erro léxico, o parser não é executado.

## 3. Definição sintática

Símbolo inicial: `programa`. Terminais em MAIÚSCULAS (tokens); `ε` é a produção vazia.

```
programa   → tela itens
tela       → CANVAS NUM NUM
itens      → item itens | ε
item       → forma | grupo
forma      → SEGMENTO NUM NUM NUM NUM modsForma
           | CIRCULO NUM NUM NUM modsForma
           | RETANGULO NUM NUM NUM NUM modsForma
modsForma  → modForma modsForma | ε
modForma   → COR_TRACO COR | COR_FUNDO COR | ESPESSURA NUM
grupo      → GRUPO modsGrupo ABRE itens FECHA
modsGrupo  → modGrupo modsGrupo | ε
modGrupo   → ROTACAO NUM | DESLOCAMENTO NUM NUM
```

Sem recursão à esquerda; a escolha de alternativa depende só do próximo token:

| Regra | Escolha |
|---|---|
| `itens` | se o token é SEGMENTO, CIRCULO, RETANGULO ou GRUPO → `item itens`; senão → ε |
| `item` | GRUPO → `grupo`; os demais → `forma` |
| `forma` | pelo token inicial (SEGMENTO, CIRCULO ou RETANGULO) |
| `modsForma` | se COR_TRACO, COR_FUNDO ou ESPESSURA → `modForma modsForma`; senão → ε |
| `modsGrupo` | se ROTACAO ou DESLOCAMENTO → `modGrupo modsGrupo`; senão → ε |

Após `itens` no nível do programa, o parser exige o fim da entrada (EOF); dentro de um grupo, exige FECHA. O parser para no primeiro erro, informando linha, o que era esperado e o que foi encontrado.

## 4. AST

| Nó (`tipo`) | Campos | Significado |
|---|---|---|
| `Programa` | `tela`, `itens` | raiz; `itens` é a lista de formas e grupos |
| `Tela` | `largura`, `altura` | dimensões do desenho |
| `Segmento` | `x1`, `y1`, `x2`, `y2`, `modificadores` | reta entre dois pontos |
| `Circulo` | `cx`, `cy`, `raio`, `modificadores` | círculo |
| `Retangulo` | `x`, `y`, `largura`, `altura`, `modificadores` | retângulo |
| `Grupo` | `modificadores`, `itens` | grupo (itens podem ser formas ou outros grupos) |
| `CorTraco` | `cor` | cor do traço |
| `CorFundo` | `cor` | cor do preenchimento |
| `Espessura` | `valor` | espessura do traço |
| `Rotacao` | `angulo` | rotação do grupo |
| `Deslocamento` | `dx`, `dy` | deslocamento do grupo |

Números são `number`; cores são `string` (nome ou `#RRGGBB`). `modificadores` é uma lista na ordem em que foram escritos. Nenhum delimitador ou palavra-chave aparece na AST.

### AST do Exemplo 1

```
{
  "tipo": "Programa",
  "tela": { "tipo": "Tela", "largura": 300, "altura": 200 },
  "itens": [
    { "tipo": "Retangulo", "x": 100, "y": 100, "largura": 100, "altura": 80,
      "modificadores": [
        { "tipo": "CorFundo", "cor": "amarelo" },
        { "tipo": "CorTraco", "cor": "preto" },
        { "tipo": "Espessura", "valor": 2 }
      ] },
    { "tipo": "Segmento", "x1": 100, "y1": 100, "x2": 150, "y2": 60,
      "modificadores": [
        { "tipo": "CorTraco", "cor": "vermelho" },
        { "tipo": "Espessura", "valor": 3 }
      ] },
    { "tipo": "Segmento", "x1": 150, "y1": 60, "x2": 200, "y2": 100,
      "modificadores": [
        { "tipo": "CorTraco", "cor": "vermelho" },
        { "tipo": "Espessura", "valor": 3 }
      ] },
    { "tipo": "Grupo",
      "modificadores": [ { "tipo": "Deslocamento", "dx": 20, "dy": 20 } ],
      "itens": [
        { "tipo": "Circulo", "cx": 0, "cy": 0, "raio": 15,
          "modificadores": [ { "tipo": "CorFundo", "cor": "laranja" } ] },
        { "tipo": "Grupo",
          "modificadores": [ { "tipo": "Rotacao", "angulo": 45 } ],
          "itens": [
            { "tipo": "Segmento", "x1": 0, "y1": 0, "x2": 25, "y2": 0,
              "modificadores": [ { "tipo": "CorTraco", "cor": "#FF8800" } ] }
          ] }
      ] }
  ]
}
```

## 5. Decisões de projeto

| Situação | Decisão |
|---|---|
| Zero | `0` é aceito como número. |
| Zero à esquerda (`05`) | **Erro léxico.** NUM exige `0` isolado ou começar em 1–9; o `\b` impede casar só o `0` de `05`. O trecho inválido reportado é `05`. |
| Número com letras (`12abc`) | Erro léxico, pelo mesmo motivo. |
| Números negativos | Não existem (só inteiros sem sinal); `-5` é erro léxico. |
| Grupo vazio (`grupo { }`) | **Aceito**; gera `Grupo` com `itens: []`. |
| Programa só com `canvas` | Aceito (lista de itens pode ser vazia). |
| Atributos repetidos | **Aceitos**; todos vão para `modificadores`, na ordem escrita. Qual vale é decisão da Fase 2. |
| Ordem dos modificadores | Livre, em qualquer ordem. |
| `canvas` | Obrigatório, uma única vez e no início; ausente ou repetido é erro sintático. |

| Situação | Decisão |
|---|---|
| Comentário | Marcador `//`, sem conflito com `#` das cores hexadecimais. |
| Cor hexadecimal | Exatamente 6 dígitos, maiúsculas ou minúsculas; `#FFF` é erro léxico. |
| Maiúsculas/minúsculas | A linguagem é *case-sensitive*: `Canvas` é erro léxico. |
| Acentos | Palavras-chave sem acento; `círculo` é erro léxico. |
| Ordem das fases | Havendo erro léxico, o parser não roda e não há AST. |
| Erros | O lexer lista todos os erros léxicos; o parser para no primeiro erro sintático. |
| Significado de rotação/deslocamento | Apenas reconhecidos e registrados na AST (Fase 2). |